Academy Hub — Sesiones 10B + 11B

Chat con WebSockets, notificaciones, señales, Celery y Redis

> **Nota previa sobre la Sesión 9B:** el patrón de "ofertas, contraofertas e intercambios" del PDF original **ya está cubierto en Academy Hub** por `enrollments` (modelo `Enrollment` con estados, `reserve_place` con `select_for_update`, permisos por propietario y API REST con `approve` / `enroll` / `cancel`). No se implementa una app `offers` porque el dominio de "compraventa de productos" no aplica a una plataforma de cursos. **No hay cambios de código para la Sesión 9B.**

Índice

1. Prefacio: lo que ya existe y lo que se añade
2. Instalación de dependencias
3. Configuración de Django, Channels y Celery
4. App `chat` — modelos
5. App `chat` — servicios
6. App `chat` — permisos
7. App `chat` — serializers
8. App `chat` — vistas web
9. App `chat` — API REST
10. App `chat` — WebSockets (Consumers + Routing)
11. App `chat` — templates y JS
12. App `chat` — admin
13. App `notifications` — ampliación de tipos
14. App `notifications` — tareas Celery
15. App `notifications` — señales
16. Router de la API
17. Migraciones
18. Tests — chat
19. Tests — notifications
20. Verificación final
21. Checklist
22. Registro de sesión

---

1. Prefacio: lo que ya existe y lo que se añade

Ya existe

- `notifications` con modelo `Notification` (`user`, `type`, `title`, `message`, `url`, `is_read`, `read_at`, `created_at`), `NotificationViewSet` y vista web.
- Redis configurado como cache.
- API REST completa en `/api/v1/`.
- `settings/base.py` con `REST_FRAMEWORK`, `SPECTACULAR_SETTINGS`, `CORS_ALLOWED_ORIGINS`, `CACHES`.
- `psycopg`, `django-cors-headers`, `drf-spectacular` y `django-filter` instalados.

Se añade en esta sesión

App `chat` (nueva):

- Modelos `Conversation`, `Participant`, `Message`.
- Servicios `create_direct_conversation`, `create_group_conversation`, `send_message`, `mark_messages_as_read`.
- Permisos `IsParticipant`.
- Serializers, ViewSet REST, vistas web, WebSocket consumer.
- Templates (listado, detalle, crear grupo) + JS mínimo.

App `notifications` (ampliación):

- Nuevos valores de `NotificationType` para eventos de dominio.
- Tareas Celery (`send_notification_email_async`, `create_notification_async`).
- Señales `post_save` para `Message`, `Enrollment`, `Scholarship`, `Course`, `Payment`.
- `apps.py` con `ready()` para registrar señales.

Configuración:

- `channels`, `channels_redis`, `celery`, `daphne` instalados.
- `config/asgi.py` reescrito para servir HTTP + WebSocket.
- `config/celery.py` creado, `config/__init__.py` modificado.
- `settings/base.py` y `settings/testing.py` ampliados.

Lo que NO se toca

- No se toca la app `enrollments`.
- No se toca la app `offers` (no existe).
- No se duplica el modelo `Notification` existente: se amplía.
- No se rompen los tests actuales.

---

2. Instalación de dependencias

Acción: ejecutar en la terminal con el entorno virtual activo.

```
pip install channels channels-redis daphne celery
pip freeze > requirements.txt
```

Qué hace cada una:

- `channels`: protocolo ASGI para WebSockets.
- `channels-redis`: Channel Layer con Redis.
- `daphne`: servidor ASGI de desarrollo (incluye `runserver` con WebSockets).
- `celery`: cola de tareas asíncronas.

---

3. Configuración de Django, Channels y Celery

3.1. `config/settings/base.py`

Archivo: `config/settings/base.py`.

Acción: modificar. Añadir al principio los imports si no están, y añadir las nuevas apps y configuraciones.

A. Añadir a `THIRD_PARTY_APPS`:

```
THIRD_PARTY_APPS = [
    "rest_framework",
    "django_filters",
    "drf_spectacular",
    "corsheaders",
    "channels",
]
```

B. Añadir a `LOCAL_APPS`:

```
LOCAL_APPS = [
    "core",
    "accounts",
    "academies",
    "courses",
    "enrollments",
    "reviews",
    "payments",
```

```
    "notifications",
    "evaluations",
    "talent",
    "level_tests",
    "search",
    "dashboard",
    "api",
    "chat",
]
```

C. Añadir `ASGI_APPLICATION` (junto a `WSGI_APPLICATION`):

```
ASGI_APPLICATION = "config.asgi.application"
```

D. Añadir al final del archivo la configuración de Channels y Celery:

```
# ============================================================
# CHANNELS / WEBSOCKETS
# ============================================================

import os

CHANNEL_LAYERS = {
    "default": {
        "BACKEND": "channels_redis.core.RedisChannelLayer",
        "CONFIG": {
            "hosts": [
                os.getenv(
                    "CHANNELS_REDIS_URL",
                    os.getenv("REDIS_URL", "redis://127.0.0.1:6379/0"),
                )
            ],
        },
    },
}


# ============================================================
# CELERY
# ============================================================

CELERY_BROKER_URL = os.getenv(
    "CELERY_BROKER_URL",
    os.getenv("REDIS_URL", "redis://127.0.0.1:6379/0"),
)

CELERY_RESULT_BACKEND = os.getenv(
    "CELERY_RESULT_BACKEND",
    os.getenv("REDIS_URL", "redis://127.0.0.1:6379/0"),
```

```
)

CELERY_ACCEPT_CONTENT = ["json"]
CELERY_TASK_SERIALIZER = "json"
CELERY_RESULT_SERIALIZER = "json"
CELERY_TIMEZONE = TIME_ZONE
```

3.2. `config/settings/testing.py`

Archivo: `config/settings/testing.py`.

Acción: modificar. Añadir al final:

```
# ============================================================
# CHANNELS EN TESTING (in-memory, sin Redis)
# ============================================================

CHANNEL_LAYERS = {
    "default": {
        "BACKEND": "channels.layers.InMemoryChannelLayer",
    },
}


# ============================================================
# CELERY EN TESTING (eager, sin worker)
# ============================================================

CELERY_TASK_ALWAYS_EAGER = True
CELERY_TASK_EAGER_PROPAGATES = True
```

3.3. `config/celery.py`

Archivo: `config/celery.py` (crear).

Acción: crear archivo nuevo con este contenido completo:

```
import os

from celery import Celery

os.environ.setdefault(
    "DJANGO_SETTINGS_MODULE",
    "config.settings.development",
)

app = Celery("AcademyHub")
```

```
app.config_from_object("django.conf:settings", namespace="CELERY")
app.autodiscover_tasks()
```

3.4. `config/__init__.py`

Archivo: `config/__init__.py`.

Acción: reemplazar entero por:

```
from .celery import app as celery_app

__all__ = ("celery_app",)
```

3.5. `config/asgi.py`

Archivo: `config/asgi.py`.

Acción: reemplazar entero por:

```
import os

from channels.auth import AuthMiddlewareStack
from channels.routing import ProtocolTypeRouter, URLRouter
from channels.security.websocket import AllowedHostsOriginValidator
from django.core.asgi import get_asgi_application

os.environ.setdefault(
    "DJANGO_SETTINGS_MODULE",
    "config.settings.development",
)

django_asgi_app = get_asgi_application()

from chat.routing import websocket_urlpatterns  # noqa: E402

application = ProtocolTypeRouter(
    {
        "http": django_asgi_app,
        "websocket": AllowedHostsOriginValidator(
            AuthMiddlewareStack(URLRouter(websocket_urlpatterns))
        ),
    }
)
```

Nota: el `import` de `chat.routing` va **después** de `get_asgi_application()` porque necesita que Django esté inicializado antes de tocar modelos o rutas.

---

4. App `chat` — modelos

4.1. Crear la app

Acción: en terminal.

```
python manage.py startapp chat
```

4.2. `chat/models.py`

Archivo: `chat/models.py`.

Acción: reemplazar entero por:

```
from django.conf import settings
from django.db import models

from courses.models import Course


class Conversation(models.Model):
    """
    Conversación entre dos o más usuarios.
    - is_group=False: chat directo (2 participantes).
    - is_group=True: chat grupal creado por un profesor o una academia.
    - course: opcional, ancla la conversación a un curso.
    """

    created_by = models.ForeignKey(
        settings.AUTH_USER_MODEL,
        on_delete=models.CASCADE,
        related_name="created_conversations",
    )

    title = models.CharField(
        max_length=150,
        blank=True,
        help_text="Obligatorio para grupos. Ej: 'Alumnos de Inglés B2'.",
    )

    is_group = models.BooleanField(
        default=False,
    )

    course = models.ForeignKey(
        Course,
        on_delete=models.SET_NULL,
        null=True,
```

```
        blank=True,
        related_name="conversations",
    )

    created_at = models.DateTimeField(
        auto_now_add=True,
    )

    updated_at = models.DateTimeField(
        auto_now=True,
    )

    class Meta:
        ordering = ["-updated_at"]
        indexes = [
            models.Index(fields=["-updated_at"]),
            models.Index(fields=["is_group"]),
        ]

    def __str__(self):
        if self.is_group:
            return f"Grupo: {self.title or self.pk}"
        return f"Chat #{self.pk}"

    @property
    def display_title(self):
        if self.is_group:
            return self.title or f"Grupo #{self.pk}"
        return f"Chat #{self.pk}"


class ParticipantRole(models.TextChoices):
    STUDENT = "student", "Alumno"
    TEACHER = "teacher", "Profesor"
    ACADEMY = "academy", "Academia"


class Participant(models.Model):
    conversation = models.ForeignKey(
        Conversation,
        on_delete=models.CASCADE,
        related_name="participants",
    )

    user = models.ForeignKey(
        settings.AUTH_USER_MODEL,
        on_delete=models.CASCADE,
        related_name="chat_participations",
    )

    role = models.CharField(
```

```
        max_length=20,
        choices=ParticipantRole.choices,
        default=ParticipantRole.STUDENT,
    )

    joined_at = models.DateTimeField(
        auto_now_add=True,
    )

    class Meta:
        ordering = ["joined_at"]
        constraints = [
            models.UniqueConstraint(
                fields=["conversation", "user"],
                name="unique_participant_per_conversation",
            ),
        ]
        indexes = [
            models.Index(fields=["conversation", "user"]),
            models.Index(fields=["user"]),
        ]

    def __str__(self):
        return f"{self.user.username} en {self.conversation_id}"


class Message(models.Model):
    conversation = models.ForeignKey(
        Conversation,
        on_delete=models.CASCADE,
        related_name="messages",
    )

    sender = models.ForeignKey(
        settings.AUTH_USER_MODEL,
        on_delete=models.CASCADE,
        related_name="sent_chat_messages",
    )

    text = models.TextField(
        max_length=1000,
    )

    is_read = models.BooleanField(
        default=False,
    )

    created_at = models.DateTimeField(
        auto_now_add=True,
    )
```

```
    class Meta:
        ordering = ["created_at"]
        indexes = [
            models.Index(fields=["conversation", "created_at"]),
            models.Index(fields=["conversation", "is_read"]),
        ]

    def __str__(self):
        return f"{self.sender.username}: {self.text[:40]}"
```

---

5. App `chat` — servicios

5.1. `chat/services.py`

Archivo: `chat/services.py` (crear).

Acción: crear archivo nuevo:

```
import logging

from asgiref.sync import async_to_sync
from channels.layers import get_channel_layer
from django.contrib.auth import get_user_model
from django.core.exceptions import PermissionDenied, ValidationError
from django.db import transaction

from .models import Conversation, Message, Participant, ParticipantRole

User = get_user_model()
logger = logging.getLogger(__name__)


def _role_for_user(user):
    if user.is_staff:
        return ParticipantRole.ACADEMY
    if user.is_academy:
        return ParticipantRole.ACADEMY
    if user.is_student:
        return ParticipantRole.STUDENT
    return ParticipantRole.STUDENT


@transaction.atomic
def create_direct_conversation(user_a, user_b, course=None):
    """
    Crea (o recupera) una conversación directa entre dos usuarios.
    Reglas:
```

```
    - No se puede crear consigo mismo.
    - Si ya existe una conversación directa con el mismo par y curso, se devuelve.
    """
    if user_a == user_b:
        raise ValidationError("No puedes abrir un chat contigo mismo.")

    if course is not None and course.academy.owner == user_b:
        pass  # es academia del curso, ok

    # Buscar conversación directa existente con los mismos participantes
    existing = (
        Conversation.objects
        .filter(is_group=False)
        .filter(participants__user=user_a)
        .filter(participants__user=user_b)
        .distinct()
    )

    if course is not None:
        existing = existing.filter(course=course)

    existing = existing.first()
    if existing:
        return existing

    conversation = Conversation.objects.create(
        created_by=user_a,
        is_group=False,
        course=course,
    )
    Participant.objects.create(
        conversation=conversation,
        user=user_a,
        role=_role_for_user(user_a),
    )
    Participant.objects.create(
        conversation=conversation,
        user=user_b,
        role=_role_for_user(user_b),
    )
    return conversation


@transaction.atomic
def create_group_conversation(creator, title, participant_ids, course=None):
    """
    Crea un grupo.
    Reglas:
    - Solo profesores y academias pueden crear grupos.
    - Un profesor solo puede crear grupos de alumnos.
    - Una academia puede crear grupos de alumnos o de profesores.
```

```
    """
    if not creator.is_academy and not creator.is_teacher:
        # fallback: si no hay is_teacher, permitimos si es staff
        if not creator.is_staff:
            raise PermissionDenied(
                "Solo profesores y academias pueden crear grupos."
            )

    if not title or not title.strip():
        raise ValidationError("El grupo necesita un título.")

    if not participant_ids:
        raise ValidationError("El grupo necesita al menos un participante.")

    participants = list(
        User.objects.filter(pk__in=participant_ids).exclude(pk=creator.pk)
    )

    if len(participants) == 0:
        raise ValidationError("No has seleccionado participantes válidos.")

    conversation = Conversation.objects.create(
        created_by=creator,
        title=title.strip(),
        is_group=True,
        course=course,
    )

    Participant.objects.create(
        conversation=conversation,
        user=creator,
        role=_role_for_user(creator),
    )

    for participant in participants:
        Participant.objects.create(
            conversation=conversation,
            user=participant,
            role=_role_for_user(participant),
        )

    return conversation


@transaction.atomic
def send_message(user, conversation_id, text):
    """
    Guarda el mensaje y emite al Channel Layer para WebSockets.
    Toda la lógica de envío pasa por aquí (REST y WebSocket).
    """
    clean_text = (text or "").strip()
```

```
    if not clean_text:
        raise ValidationError("El mensaje no puede estar vacío.")

    if len(clean_text) > 1000:
        raise ValidationError("El mensaje no puede superar los 1000 caracteres.")

    conversation = (
        Conversation.objects
        .select_for_update()
        .get(pk=conversation_id)
    )

    is_participant = Participant.objects.filter(
        conversation=conversation,
        user=user,
    ).exists()

    if not is_participant:
        raise PermissionDenied("No participas en esta conversación.")

    message = Message.objects.create(
        conversation=conversation,
        sender=user,
        text=clean_text,
    )

    conversation.save(update_fields=["updated_at"])

    # Emitir al Channel Layer
    try:
        channel_layer = get_channel_layer()
        if channel_layer is not None:
            async_to_sync(channel_layer.group_send)(
                f"chat_{conversation.pk}",
                {
                    "type": "chat_message",
                    "message_id": message.pk,
                    "sender": message.sender.username,
                    "text": message.text,
                    "created_at": message.created_at.isoformat(),
                },
            )
    except Exception:
        logger.exception(
            "No se pudo emitir al Channel Layer para conversación %s",
            conversation_id,
        )

    return message
```

```
@transaction.atomic
def mark_messages_as_read(conversation_id, user):
    conversation = Conversation.objects.get(pk=conversation_id)

    is_participant = Participant.objects.filter(
        conversation=conversation,
        user=user,
    ).exists()

    if not is_participant:
        raise PermissionDenied("No participas en esta conversación.")

    Message.objects.filter(
        conversation=conversation,
        is_read=False,
    ).exclude(sender=user).update(is_read=True)
```

Nota: el atributo `is_teacher` en `User` no existe. He puesto un `fallback` que permite staff. Si en el futuro se añade `is_teacher` al `User`, el código ya lo usará. Si no, no rompe: los profesores con `is_academy=True` pueden crear grupos. Si quieres que un profesor "no academia" pueda crear grupos, hay que añadir `is_teacher` al modelo `User` en una sesión futura.

---

6. App `chat` — permisos

6.1. `chat/permissions.py`

Archivo: `chat/permissions.py` (crear).

Acción: crear archivo nuevo:

```
from rest_framework.permissions import SAFE_METHODS, BasePermission

from .models import Participant


class IsParticipant(BasePermission):
    """
    Solo los participantes de la conversación pueden verla o escribir en ella.
    """

    def has_permission(self, request, view):
        return request.user.is_authenticated

    def has_object_permission(self, request, view, obj):
        return Participant.objects.filter(
```

```
            conversation=obj,
            user=request.user,
        ).exists()
```

---

7. App `chat` — serializers

7.1. `chat/serializers.py`

Archivo: `chat/serializers.py` (crear).

Acción: crear archivo nuevo:

```
from rest_framework import serializers

from .models import Conversation, Message, Participant


class ParticipantSerializer(serializers.ModelSerializer):
    username = serializers.CharField(source="user.username", read_only=True)
    role_display = serializers.CharField(source="get_role_display", read_only=True)

    class Meta:
        model = Participant
        fields = ["id", "user", "username", "role", "role_display", "joined_at"]
        read_only_fields = fields


class MessageSerializer(serializers.ModelSerializer):
    sender_username = serializers.CharField(source="sender.username", read_only=True)

    class Meta:
        model = Message
        fields = [
            "id",
            "conversation",
            "sender",
            "sender_username",
            "text",
            "is_read",
            "created_at",
        ]
        read_only_fields = fields


class ConversationSerializer(serializers.ModelSerializer):
    participants = ParticipantSerializer(many=True, read_only=True)
    created_by_username = serializers.CharField(
```

```
        source="created_by.username", read_only=True
    )
    course_title = serializers.CharField(source="course.title", read_only=True)
    last_message = serializers.SerializerMethodField()
    unread_count = serializers.SerializerMethodField()

    class Meta:
        model = Conversation
        fields = [
            "id",
            "created_by",
            "created_by_username",
            "title",
            "is_group",
            "course",
            "course_title",
            "created_at",
            "updated_at",
            "participants",
            "last_message",
            "unread_count",
        ]
        read_only_fields = fields

    def get_last_message(self, obj):
        last = obj.messages.last()
        return last.text if last else None

    def get_unread_count(self, obj):
        user = self.context["request"].user
        return obj.messages.filter(is_read=False).exclude(sender=user).count()


class SendMessageInputSerializer(serializers.Serializer):
    text = serializers.CharField(max_length=1000, allow_blank=False)


class CreateDirectConversationInputSerializer(serializers.Serializer):
    username = serializers.CharField()
    course = serializers.IntegerField(required=False, allow_null=True)


class CreateGroupConversationInputSerializer(serializers.Serializer):
    title = serializers.CharField(max_length=150)
    participants = serializers.ListField(
        child=serializers.IntegerField(),
        allow_empty=False,
    )
    course = serializers.IntegerField(required=False, allow_null=True)
```

8. App `chat` — vistas web

8.1. `chat/views_web.py`

Archivo: `chat/views_web.py` (crear). **Separado** de `views.py` (REST) para no mezclar.

Acción: crear archivo nuevo:

```
from django.contrib import messages
from django.contrib.auth import get_user_model
from django.contrib.auth.mixins import LoginRequiredMixin
from django.core.exceptions import PermissionDenied, ValidationError
from django.shortcuts import get_object_or_404, redirect, render
from django.views import View
from django.views.generic import ListView

from .models import Conversation, Participant
from .services import (
    create_direct_conversation,
    create_group_conversation,
    mark_messages_as_read,
    send_message,
)

User = get_user_model()


class ConversationListView(LoginRequiredMixin, ListView):
    model = Conversation
    template_name = "chat/conversation_list.html"
    context_object_name = "conversations"

    def get_queryset(self):
        return (
            Conversation.objects
            .filter(participants__user=self.request.user)
            .prefetch_related("participants__user")
            .distinct()
            .order_by("-updated_at")
        )


class ConversationDetailView(LoginRequiredMixin, View):
    template_name = "chat/conversation_detail.html"

    def get(self, request, pk):
        conversation = get_object_or_404(Conversation, pk=pk)

        if not Participant.objects.filter(
```

```
            conversation=conversation, user=request.user
        ).exists():
            raise PermissionDenied("No participas en esta conversación.")

        mark_messages_as_read(conversation.pk, request.user)

        messages_qs = (
            conversation.messages
            .select_related("sender")
            .order_by("created_at")
        )

        return render(
            request,
            self.template_name,
            {
                "conversation": conversation,
                "messages": messages_qs,
                "is_group": conversation.is_group,
            },
        )

    def post(self, request, pk):
        conversation = get_object_or_404(Conversation, pk=pk)

        try:
            send_message(request.user, conversation.pk, request.POST.get("text", ""))
            messages.success(request, "Mensaje enviado.")
        except (ValidationError, PermissionDenied) as e:
            messages.error(request, str(e))

        return redirect("chat:detail", pk=pk)


class CreateGroupView(LoginRequiredMixin, View):
    template_name = "chat/create_group.html"

    def get(self, request):
        return render(request, self.template_name)

    def post(self, request):
        title = request.POST.get("title", "").strip()
        usernames = request.POST.getlist("participants")
        participant_ids = []

        for username in usernames:
            try:
                u = User.objects.get(username=username)
                participant_ids.append(u.pk)
            except User.DoesNotExist:
                messages.warning(
```

```
                    request, f"Usuario '{username}' no encontrado."
                )

        try:
            conversation = create_group_conversation(
                creator=request.user,
                title=title,
                participant_ids=participant_ids,
            )
            messages.success(request, "Grupo creado.")
            return redirect("chat:detail", pk=conversation.pk)
        except (ValidationError, PermissionDenied) as e:
            messages.error(request, str(e))
            return render(request, self.template_name)


class CreateDirectConversationView(LoginRequiredMixin, View):

    def post(self, request):
        username = request.POST.get("username", "").strip()

        try:
            other = User.objects.get(username=username)
        except User.DoesNotExist:
            messages.error(request, f"Usuario '{username}' no encontrado.")
            return redirect("chat:list")

        try:
            conversation = create_direct_conversation(
                user_a=request.user,
                user_b=other,
            )
            return redirect("chat:detail", pk=conversation.pk)
        except (ValidationError, PermissionDenied) as e:
            messages.error(request, str(e))
            return redirect("chat:list")
```

8.2. `chat/urls.py`

Archivo: `chat/urls.py` (crear).

Acción: crear archivo nuevo:

```
from django.urls import path

from . import views_web

app_name = "chat"
```

```
urlpatterns = [
    path("", views_web.ConversationListView.as_view(), name="list"),
    path("<int:pk>/", views_web.ConversationDetailView.as_view(), name="detail"),
    path("create/group/", views_web.CreateGroupView.as_view(), name="create_group"),
    path(
        "create/direct/",
        views_web.CreateDirectConversationView.as_view(),
        name="create_direct",
    ),
]
```

8.3. `config/urls.py`

Archivo: `config/urls.py`.

Acción: modificar. Añadir el `include` del chat. Busca el bloque `urlpatterns = [...]` y añade dentro:

```
path("chat/", include("chat.urls")),
```

Debe quedar al mismo nivel que los otros `include`. Por ejemplo:

```
urlpatterns = [
    path("admin/", admin.site.urls),

    path("", include("core.urls")),
    path("accounts/", include("accounts.urls")),
    path("academies/", include("academies.urls")),
    path("courses/", include("courses.urls")),
    path("enrollments/", include("enrollments.urls")),
    path("reviews/", include("reviews.urls")),
    path("payments/", include("payments.urls")),
    path("notifications/", include("notifications.urls")),
    path("evaluations/", include("evaluations.urls")),
    path("talent/", include("talent.urls")),
    path("level-tests/", include("level_tests.urls")),
    path("search/", include("search.urls")),
    path("dashboard/", include("dashboard.urls")),
    path("chat/", include("chat.urls")),

    path("api/v1/", include("api.urls")),
    path("api/schema/", SpectacularAPIView.as_view(), name="schema"),
    path(
        "api/docs/",
        SpectacularSwaggerView.as_view(url_name="schema"),
        name="swagger-ui",
    ),
    path(
```

```
        "api/redoc/",
        SpectacularRedocView.as_view(url_name="schema"),
        name="redoc",
    ),
]
```

---

9. App `chat` — API REST

9.1. `chat/views.py`

Archivo: `chat/views.py` (crear).

Acción: crear archivo nuevo:

```
from django.contrib.auth import get_user_model
from django.core.exceptions import PermissionDenied, ValidationError
from django.db.models import Q
from rest_framework import status, viewsets
from rest_framework.decorators import action
from rest_framework.permissions import IsAuthenticated
from rest_framework.response import Response

from .models import Conversation
from .permissions import IsParticipant
from .serializers import (
    ConversationSerializer,
    CreateDirectConversationInputSerializer,
    CreateGroupConversationInputSerializer,
    MessageSerializer,
    SendMessageInputSerializer,
)
from .services import (
    create_direct_conversation,
    create_group_conversation,
    mark_messages_as_read,
    send_message,
)

User = get_user_model()


class ConversationViewSet(viewsets.ReadOnlyModelViewSet):
    serializer_class = ConversationSerializer
    permission_classes = [IsAuthenticated, IsParticipant]

    def get_queryset(self):
        user = self.request.user
```

```
        return (
            Conversation.objects
            .filter(Q(participants__user=user))
            .prefetch_related("participants__user", "messages")
            .distinct()
            .order_by("-updated_at")
        )

    @action(detail=True, methods=["get"])
    def messages(self, request, pk=None):
        conversation = self.get_object()
        mark_messages_as_read(conversation.pk, request.user)
        qs = conversation.messages.select_related("sender").order_by("created_at")
        return Response(MessageSerializer(qs, many=True).data)

    @action(detail=True, methods=["post"])
    def send(self, request, pk=None):
        conversation = self.get_object()
        serializer = SendMessageInputSerializer(data=request.data)
        serializer.is_valid(raise_exception=True)

        try:
            msg = send_message(
                user=request.user,
                conversation_id=conversation.pk,
                text=serializer.validated_data["text"],
            )
            return Response(
                MessageSerializer(msg).data,
                status=status.HTTP_201_CREATED,
            )
        except (ValidationError, PermissionDenied) as e:
            return Response(
                {"detail": str(e)},
                status=status.HTTP_400_BAD_REQUEST,
            )

    @action(detail=True, methods=["post"])
    def mark_read(self, request, pk=None):
        conversation = self.get_object()
        mark_messages_as_read(conversation.pk, request.user)
        return Response({"status": "ok"}, status=status.HTTP_200_OK)

    @action(detail=False, methods=["post"])
    def create_direct(self, request):
        serializer = CreateDirectConversationInputSerializer(data=request.data)
        serializer.is_valid(raise_exception=True)

        try:
            other = User.objects.get(
                username=serializer.validated_data["username"]
```

```
            )
        except User.DoesNotExist:
            return Response(
                {"detail": "Usuario no encontrado."},
                status=status.HTTP_404_NOT_FOUND,
            )

        try:
            conversation = create_direct_conversation(
                user_a=request.user,
                user_b=other,
            )
            return Response(
                ConversationSerializer(
                    conversation, context={"request": request}
                ).data,
                status=status.HTTP_201_CREATED,
            )
        except (ValidationError, PermissionDenied) as e:
            return Response(
                {"detail": str(e)},
                status=status.HTTP_400_BAD_REQUEST,
            )

    @action(detail=False, methods=["post"])
    def create_group(self, request):
        serializer = CreateGroupConversationInputSerializer(data=request.data)
        serializer.is_valid(raise_exception=True)

        try:
            conversation = create_group_conversation(
                creator=request.user,
                title=serializer.validated_data["title"],
                participant_ids=serializer.validated_data["participants"],
            )
            return Response(
                ConversationSerializer(
                    conversation, context={"request": request}
                ).data,
                status=status.HTTP_201_CREATED,
            )
        except (ValidationError, PermissionDenied) as e:
            return Response(
                {"detail": str(e)},
                status=status.HTTP_400_BAD_REQUEST,
            )
```

10. App `chat` — WebSockets (Consumers + Routing)

10.1. `chat/consumers.py`

Archivo: `chat/consumers.py` (crear).

Acción: crear archivo nuevo:

```
import logging

from channels.db import database_sync_to_async
from channels.generic.websocket import AsyncJsonWebsocketConsumer
from django.core.exceptions import PermissionDenied, ValidationError

from .models import Participant
from .services import send_message

logger = logging.getLogger(__name__)


class ChatConsumer(AsyncJsonWebsocketConsumer):

    async def connect(self):
        self.user = self.scope["user"]

        if not self.user.is_authenticated:
            await self.close()
            return

        self.conversation_id = self.scope["url_route"]["kwargs"]["conversation_id"]

        if not await self._is_participant():
            await self.close()
            return

        self.room_group_name = f"chat_{self.conversation_id}"
        await self.channel_layer.group_add(
            self.room_group_name,
            self.channel_name,
        )
        await self.accept()

    async def disconnect(self, close_code):
        if hasattr(self, "room_group_name"):
            await self.channel_layer.group_discard(
                self.room_group_name,
                self.channel_name,
            )

    async def receive_json(self, content, **kwargs):
        text = content.get("text", "").strip()
        if not text:
```

```
            return

        try:
            message = await database_sync_to_async(send_message)(
                self.user,
                self.conversation_id,
                text,
            )
        except (ValidationError, PermissionDenied) as e:
            await self.send_json({"type": "error", "detail": str(e)})
            return

        # El mensaje se emite desde send_message al Channel Layer.
        # Aquí no volvemos a emitir para no duplicar.

    async def chat_message(self, event):
        await self.send_json(
            {
                "type": "message",
                "message_id": event["message_id"],
                "sender": event["sender"],
                "text": event["text"],
                "created_at": event["created_at"],
            }
        )

    @database_sync_to_async
    def _is_participant(self):
        return Participant.objects.filter(
            conversation_id=self.conversation_id,
            user=self.user,
        ).exists()
```

10.2. `chat/routing.py`

Archivo: `chat/routing.py` (crear).

Acción: crear archivo nuevo:

```
from django.urls import re_path

from .consumers import ChatConsumer

websocket_urlpatterns = [
    re_path(
        r"^ws/chat/(?P<conversation_id>\d+)/$",
        ChatConsumer.as_asgi(),
    ),
]
```

11. App `chat` — templates y JS

11.1. `templates/chat/conversation_list.html`

Archivo: `templates/chat/conversation_list.html` (crear).

Acción: crear archivo nuevo:

```
{% extends "base.html" %}

{% block title %}
Mensajes | Academy Hub
{% endblock %}

{% block content %}

<h1>Mensajes</h1>

<div class="chat-actions">

    <form method="post" action="{% url 'chat:create_direct' %}">
        {% csrf_token %}
        <input type="text" name="username" placeholder="Usuario (para chat directo)">
        <button type="submit">Abrir chat directo</button>
    </form>

    {% if user.is_academy or user.is_staff %}
        <a href="{% url 'chat:create_group' %}">Crear grupo</a>
    {% endif %}

</div>

<ul class="conversation-list">

    {% for conversation in conversations %}

        <li>
            <a href="{% url 'chat:detail' conversation.pk %}">
                {{ conversation.display_title }}
            </a>
            <small>
                Actualizado: {{ conversation.updated_at }}
            </small>
        </li>

    {% empty %}

        <li>No tienes conversaciones todavía.</li>
```

```
    {% endfor %}

</ul>

{% endblock %}
```

11.2. `templates/chat/conversation_detail.html`

Archivo: `templates/chat/conversation_detail.html` (crear).

Acción: crear archivo nuevo:

```
{% extends "base.html" %}

{% block title %}
{{ conversation.display_title }} | Academy Hub
{% endblock %}

{% block content %}

<h1>{{ conversation.display_title }}</h1>

{% if conversation.course %}
    <p>
        Curso:
        <a href="{{ conversation.course.get_absolute_url }}">
            {{ conversation.course.title }}
        </a>
    </p>
{% endif %}

<p>
    Participantes:
    {% for p in conversation.participants.all %}
        {{ p.user.username }} ({{ p.get_role_display }}){% if not forloop.last %}, {% endif %}
    {% endfor %}
</p>

<hr>

<div id="chat-root" data-conversation-id="{{ conversation.pk }}">

    <div id="messages-list">
        {% for message in messages %}
            <article>
                <p>
                    <strong>{{ message.sender.username }}</strong>
                    — {{ message.created_at }}
```

```
                </p>
                <p>{{ message.text }}</p>
            </article>
            <hr>
        {% empty %}
            <p>Todavía no hay mensajes.</p>
        {% endfor %}
    </div>

    <form id="message-form" method="post" action="{% url 'chat:detail' conversation.pk %}">
        {% csrf_token %}
        <textarea id="message-text" name="text" rows="2" placeholder="Escribe un mensaje..."></te
        <button type="submit">Enviar</button>
    </form>

</div>

{% endblock %}

{% block scripts %}
<script src="/static/chat/js/chat.js"></script>
{% endblock %}
```

Nota: el bloque `{% block scripts %}` tiene que existir en `base.html`. Si no existe, añádelo al final de `base.html`, antes de `</body>`:

```
{% block scripts %}{% endblock %}
```

11.3. `templates/chat/create_group.html`

Archivo: `templates/chat/create_group.html` (crear).

Acción: crear archivo nuevo:

```
{% extends "base.html" %}

{% block title %}
Crear grupo | Academy Hub
{% endblock %}

{% block content %}

<h1>Crear grupo</h1>

<form method="post">
    {% csrf_token %}

    <p>
```

```
        <label>Título</label>
        <input type="text" name="title" required maxlength="150">
    </p>

    <p>
        <label>Usuarios (uno por línea)</label>
        <textarea name="participants" rows="6"
            placeholder="laura&#10;carlos&#10;ana"></textarea>
    </p>

    <button type="submit">Crear grupo</button>
</form>

{% endblock %}
```

11.4. `static/chat/js/chat.js`

Archivo: `static/chat/js/chat.js` (crear). **Nota:** el directorio `static/chat/js/` hay que crearlo.

Acción: crear archivo nuevo:

```
(function () {
    const root = document.getElementById("chat-root");
    if (!root) return;

    const conversationId = root.dataset.conversationId;
    const protocol = window.location.protocol === "https:" ? "wss" : "ws";
    const wsUrl = `${protocol}://${window.location.host}/ws/chat/${conversationId}/`;

    const ws = new WebSocket(wsUrl);
    const list = document.getElementById("messages-list");
    const form = document.getElementById("message-form");
    const input = document.getElementById("message-text");

    ws.onopen = function () {
        console.log("WebSocket conectado");
    };

    ws.onmessage = function (event) {
        const data = JSON.parse(event.data);
        if (data.type !== "message") return;

        const article = document.createElement("article");
        const header = document.createElement("p");
        const sender = document.createElement("strong");
        sender.textContent = data.sender;
        header.appendChild(sender);
        header.appendChild(document.createTextNode(" — " + data.created_at));
```

```
        const body = document.createElement("p");
        body.textContent = data.text;

        article.appendChild(header);
        article.appendChild(body);
        list.appendChild(article);
        list.appendChild(document.createElement("hr"));

        list.scrollTop = list.scrollHeight;
    };

    form.addEventListener("submit", function (event) {
        event.preventDefault();
        const text = input.value.trim();
        if (!text) return;

        if (ws.readyState === WebSocket.OPEN) {
            ws.send(JSON.stringify({ text: text }));
            input.value = "";
        } else {
            // Fallback a POST si el socket no está abierto
            form.submit();
        }
    });
})();
```

---

12. App `chat` — admin

12.1. `chat/admin.py`

Archivo: `chat/admin.py`.

Acción: reemplazar entero por:

```
from django.contrib import admin

from .models import Conversation, Message, Participant


class ParticipantInline(admin.TabularInline):
    model = Participant
    extra = 0
    autocomplete_fields = ["user"]


class MessageInline(admin.TabularInline):
    model = Message
```

```
    extra = 0
    readonly_fields = ["sender", "text", "is_read", "created_at"]
    can_delete = False


@admin.register(Conversation)
class ConversationAdmin(admin.ModelAdmin):
    list_display = ["id", "display_title", "is_group", "course", "created_at", "updated_at"]
    list_filter = ["is_group", "created_at"]
    search_fields = ["title", "course__title"]
    autocomplete_fields = ["created_by", "course"]
    inlines = [ParticipantInline, MessageInline]


@admin.register(Message)
class MessageAdmin(admin.ModelAdmin):
    list_display = ["id", "conversation", "sender", "is_read", "created_at"]
    list_filter = ["is_read", "created_at"]
    search_fields = ["sender__username", "text"]
    autocomplete_fields = ["conversation", "sender"]
```

---

13. App `notifications` — ampliación de tipos

13.1. `notifications/models.py`

Archivo: `notifications/models.py`.

Acción: reemplazar entero por:

```
from django.conf import settings
from django.db import models


class Notification(models.Model):

    class Type(models.TextChoices):
        INFO = "info", "Información"
        WARNING = "warning", "Aviso"
        SUCCESS = "success", "Éxito"
        REMINDER = "reminder", "Recordatorio"
        NEW_MESSAGE = "new_message", "Nuevo mensaje de chat"
        ENROLLMENT_RECEIVED = "enrollment_received", "Solicitud de inscripción"
        ENROLLMENT_APPROVED = "enrollment_approved", "Inscripción aprobada"
        ENROLLMENT_REJECTED = "enrollment_rejected", "Inscripción rechazada"
        SCHOLARSHIP_APPROVED = "scholarship_approved", "Beca aprobada"
        SCHOLARSHIP_REJECTED = "scholarship_rejected", "Beca rechazada"
        COURSE_PUBLISHED = "course_published", "Curso publicado"
```

```
        SYSTEM = "system", "Aviso del sistema"

    user = models.ForeignKey(
        settings.AUTH_USER_MODEL,
        on_delete=models.CASCADE,
        related_name="notifications",
    )

    type = models.CharField(
        max_length=30,
        choices=Type.choices,
        default=Type.INFO,
    )

    title = models.CharField(max_length=200)
    message = models.TextField()
    url = models.CharField(max_length=300, blank=True)
    is_read = models.BooleanField(default=False)
    read_at = models.DateTimeField(null=True, blank=True)

    created_at = models.DateTimeField(auto_now_add=True)

    class Meta:
        ordering = ["-created_at"]
        indexes = [
            models.Index(fields=["user", "is_read"]),
            models.Index(fields=["user", "is_read", "-created_at"]),
        ]

    def __str__(self):
        return f"{self.user.username} — {self.title}"
```

---

14. App `notifications` — tareas Celery

14.1. `notifications/tasks.py`

Archivo: `notifications/tasks.py` (crear).

Acción: crear archivo nuevo:

```
import logging

from celery import shared_task
from django.contrib.auth import get_user_model

from .models import Notification
```

```python
logger = logging.getLogger(__name__)
User = get_user_model()


@shared_task
def send_notification_email_async(user_id, subject, body):
    """
    Simula envío de email en segundo plano.
    En producción: usar django.core.mail.send_mail.
    """
    try:
        user = User.objects.get(pk=user_id)
    except User.DoesNotExist:
        logger.error("send_notification_email_async: user %s not found", user_id)
        return "user_not_found"

    logger.info(
        "Enviando email a %s | Asunto: %s | Cuerpo: %s",
        user.email,
        subject,
        body[:80],
    )
    return f"email_sent_for_user_{user_id}"


@shared_task
def create_notification_async(user_id, notification_type, title, message, url=""):
    """
    Crea la notificación in-app y dispara el email en otra tarea.
    """
    try:
        user = User.objects.get(pk=user_id)
    except User.DoesNotExist:
        logger.error("create_notification_async: user %s not found", user_id)
        return "user_not_found"

    notification = Notification.objects.create(
        user=user,
        type=notification_type,
        title=title,
        message=message,
        url=url,
    )

    send_notification_email_async.delay(
        user_id=user_id,
        subject=title,
        body=message,
    )

    return f"notification_{notification.pk}_created"
```

15. App `notifications` — señales

15.1. `notifications/signals.py`

Archivo: `notifications/signals.py` (crear).

Acción: crear archivo nuevo:

```
from django.db.models.signals import post_save, pre_save
from django.dispatch import receiver

from chat.models import Message
from courses.models import Course
from enrollments.models import Enrollment
from payments.models import Payment, Scholarship

from .models import Notification
from .tasks import create_notification_async


# ============================================================
# CHAT — nuevo mensaje
# ============================================================

@receiver(post_save, sender=Message)
def notify_new_message(sender, instance, created, **kwargs):
    if not created:
        return

    conversation = instance.conversation

    for participant in conversation.participants.exclude(user=instance.sender):
        create_notification_async.delay(
            user_id=participant.user_id,
            notification_type=Notification.Type.NEW_MESSAGE,
            title="Nuevo mensaje de chat",
            message=f"{instance.sender.username}: {instance.text[:80]}",
            url=f"/chat/{conversation.pk}/",
        )


# ============================================================
# ENROLLMENT — cambios de estado
# ============================================================

@receiver(pre_save, sender=Enrollment)
```

```
def enrollment_pre_save(sender, instance, **kwargs):
    if instance.pk:
        try:
            previous = Enrollment.objects.get(pk=instance.pk)
            instance._old_status = previous.status
        except Enrollment.DoesNotExist:
            instance._old_status = None
    else:
        instance._old_status = None


@receiver(post_save, sender=Enrollment)
def enrollment_post_save(sender, instance, created, **kwargs):
    academy_user_id = instance.convocatoria.course.academy.owner_id

    if created:
        create_notification_async.delay(
            user_id=academy_user_id,
            notification_type=Notification.Type.ENROLLMENT_RECEIVED,
            title="Nueva solicitud de inscripción",
            message=(
                f"{instance.student.username} ha solicitado plaza en "
                f"{instance.convocatoria.course.title}."
            ),
            url=f"/enrollments/{instance.pk}/",
        )
        return

    old = getattr(instance, "_old_status", None)

    if old is None or old == instance.status:
        return

    if instance.status == "approved":
        create_notification_async.delay(
            user_id=instance.student_id,
            notification_type=Notification.Type.ENROLLMENT_APPROVED,
            title="Inscripción aprobada",
            message=(
                f"Tu inscripción en {instance.convocatoria.course.title} "
                f"ha sido aprobada."
            ),
            url=f"/enrollments/{instance.pk}/",
        )
    elif instance.status == "rejected":
        create_notification_async.delay(
            user_id=instance.student_id,
            notification_type=Notification.Type.ENROLLMENT_REJECTED,
            title="Inscripción rechazada",
            message=(
                f"Tu inscripción en {instance.convocatoria.course.title} "
```

```
                f"ha sido rechazada."
            ),
            url=f"/enrollments/{instance.pk}/",
        )


# ============================================================
# SCHOLARSHIP — cambios de estado
# ============================================================

@receiver(pre_save, sender=Scholarship)
def scholarship_pre_save(sender, instance, **kwargs):
    if instance.pk:
        try:
            previous = Scholarship.objects.get(pk=instance.pk)
            instance._old_status = previous.status
        except Scholarship.DoesNotExist:
            instance._old_status = None
    else:
        instance._old_status = None


@receiver(post_save, sender=Scholarship)
def scholarship_post_save(sender, instance, created, **kwargs):
    if created:
        return

    old = getattr(instance, "_old_status", None)
    if old is None or old == instance.status:
        return

    if instance.status == "approved":
        create_notification_async.delay(
            user_id=instance.student_id,
            notification_type=Notification.Type.SCHOLARSHIP_APPROVED,
            title="Beca aprobada",
            message="Tu solicitud de beca ha sido aprobada.",
            url=f"/payments/scholarships/",
        )
    elif instance.status == "rejected":
        create_notification_async.delay(
            user_id=instance.student_id,
            notification_type=Notification.Type.SCHOLARSHIP_REJECTED,
            title="Beca rechazada",
            message="Tu solicitud de beca ha sido rechazada.",
            url=f"/payments/scholarships/",
        )


# ============================================================
# COURSE — cambio a published
```

```python
# ============================================================

@receiver(pre_save, sender=Course)
def course_pre_save(sender, instance, **kwargs):
    if instance.pk:
        try:
            previous = Course.objects.get(pk=instance.pk)
            instance._old_status = previous.status
        except Course.DoesNotExist:
            instance._old_status = None
    else:
        instance._old_status = None


@receiver(post_save, sender=Course)
def course_post_save(sender, instance, created, **kwargs):
    if created:
        return

    old = getattr(instance, "_old_status", None)
    if old == instance.status:
        return

    if instance.status == "published":
        # Notificar a los usuarios que tienen la academia como favorita
        from academies.models import AcademyFavorite
        favorites = AcademyFavorite.objects.filter(academy=instance.academy)
        for favorite in favorites:
            create_notification_async.delay(
                user_id=favorite.user_id,
                notification_type=Notification.Type.COURSE_PUBLISHED,
                title="Nuevo curso publicado",
                message=(
                    f"{instance.academy.name} ha publicado "
                    f"'{instance.title}'."
                ),
                url=f"/courses/{instance.pk}/{instance.slug}/",
            )


# ============================================================
# PAYMENT — pago confirmado
# ============================================================

@receiver(pre_save, sender=Payment)
def payment_pre_save(sender, instance, **kwargs):
    if instance.pk:
        try:
            previous = Payment.objects.get(pk=instance.pk)
            instance._old_status = previous.status
        except Payment.DoesNotExist:
```

```
            instance._old_status = None
    else:
        instance._old_status = None


@receiver(post_save, sender=Payment)
def payment_post_save(sender, instance, created, **kwargs):
    old = getattr(instance, "_old_status", None)
    if old is None or old == instance.status:
        return

    if instance.status == "paid":
        create_notification_async.delay(
            user_id=instance.enrollment.student_id,
            notification_type=Notification.Type.SUCCESS,
            title="Pago confirmado",
            message=(
                f"Hemos recibido tu pago de {instance.amount} €."
            ),
            url=f"/payments/",
        )
```

15.2. `notifications/apps.py`

Archivo: `notifications/apps.py`.

Acción: reemplazar entero por:

```
from django.apps import AppConfig


class NotificationsConfig(AppConfig):
    default_auto_field = "django.db.models.BigAutoField"
    name = "notifications"

    def ready(self):
        import notifications.signals  # noqa: F401
```

---

16. Router de la API

16.1. `api/urls.py`

Archivo: `api/urls.py`.

Acción: modificar. Añadir el import del `ConversationViewSet` y el `router.register`.

A. Añadir el import:

```
from chat.views import ConversationViewSet
```

B. Añadir el registro del router, junto a los demás `router.register`:

```
router.register("conversations", ConversationViewSet, basename="conversation")
```

Debe quedar dentro del bloque donde ya están `products`, `categories`, etc.

---

17. Migraciones

Acción: en terminal.

```
python manage.py makemigrations chat
python manage.py makemigrations notifications
python manage.py migrate
python manage.py check
```

Nota: el cambio de `max_length=20` a `max_length=30` en `Notification.type` genera una migración pequeña. Es correcto.

---

18. Tests — chat

18.1. `chat/tests/__init__.py`

Archivo: `chat/tests/__init__.py` (crear). **Vacío.**

18.2. `chat/tests/test_models.py`

Archivo: `chat/tests/test_models.py` (crear).

Acción: crear archivo nuevo:

```
from django.contrib.auth import get_user_model
from django.db import IntegrityError
from django.test import TestCase

from chat.models import Conversation, Message, Participant, ParticipantRole
from courses.models import Category, Course
```

```python
User = get_user_model()


class ConversationModelTests(TestCase):

    def setUp(self):
        self.student = User.objects.create_user(
            username="laura",
            email="laura@example.com",
            password="TestPassword123!",
            is_student=True,
        )
        self.teacher = User.objects.create_user(
            username="profesor",
            email="profesor@example.com",
            password="TestPassword123!",
            is_academy=True,
        )

    def test_create_direct_conversation(self):
        conv = Conversation.objects.create(
            created_by=self.student,
            is_group=False,
        )
        self.assertFalse(conv.is_group)
        self.assertEqual(conv.created_by, self.student)

    def test_create_group_conversation(self):
        conv = Conversation.objects.create(
            created_by=self.teacher,
            title="Alumnos Inglés",
            is_group=True,
        )
        self.assertTrue(conv.is_group)
        self.assertEqual(conv.title, "Alumnos Inglés")

    def test_participant_unique_per_conversation(self):
        conv = Conversation.objects.create(created_by=self.student)
        Participant.objects.create(conversation=conv, user=self.student)
        with self.assertRaises(IntegrityError):
            Participant.objects.create(conversation=conv, user=self.student)

    def test_participant_role_default(self):
        conv = Conversation.objects.create(created_by=self.student)
        p = Participant.objects.create(conversation=conv, user=self.student)
        self.assertEqual(p.role, ParticipantRole.STUDENT)


class MessageModelTests(TestCase):

    def setUp(self):
```

```
        self.user = User.objects.create_user(
            username="laura",
            email="laura@example.com",
            password="TestPassword123!",
            is_student=True,
        )
        self.conv = Conversation.objects.create(created_by=self.user)
        Participant.objects.create(conversation=self.conv, user=self.user)

    def test_create_message(self):
        msg = Message.objects.create(
            conversation=self.conv,
            sender=self.user,
            text="Hola",
        )
        self.assertEqual(msg.text, "Hola")
        self.assertFalse(msg.is_read)

    def test_message_ordering(self):
        m1 = Message.objects.create(
            conversation=self.conv, sender=self.user, text="Primero"
        )
        m2 = Message.objects.create(
            conversation=self.conv, sender=self.user, text="Segundo"
        )
        msgs = list(Message.objects.all())
        self.assertEqual(msgs[0], m1)
        self.assertEqual(msgs[1], m2)
```

18.3. `chat/tests/test_services.py`

Archivo: `chat/tests/test_services.py` (crear).

Acción: crear archivo nuevo:

```
from django.contrib.auth import get_user_model
from django.core.exceptions import PermissionDenied, ValidationError
from django.test import TestCase

from chat.models import Participant
from chat.services import (
    create_direct_conversation,
    create_group_conversation,
    mark_messages_as_read,
    send_message,
)

User = get_user_model()
```

```python
class DirectConversationServiceTests(TestCase):

    def setUp(self):
        self.student = User.objects.create_user(
            username="laura",
            email="laura@example.com",
            password="TestPassword123!",
            is_student=True,
        )
        self.teacher = User.objects.create_user(
            username="profesor",
            email="profesor@example.com",
            password="TestPassword123!",
            is_academy=True,
        )

    def test_create_direct_conversation(self):
        conv = create_direct_conversation(self.student, self.teacher)
        self.assertEqual(conv.participants.count(), 2)
        self.assertFalse(conv.is_group)

    def test_create_direct_twice_returns_same(self):
        c1 = create_direct_conversation(self.student, self.teacher)
        c2 = create_direct_conversation(self.student, self.teacher)
        self.assertEqual(c1.pk, c2.pk)

    def test_cannot_create_with_self(self):
        with self.assertRaises(ValidationError):
            create_direct_conversation(self.student, self.student)


class GroupConversationServiceTests(TestCase):

    def setUp(self):
        self.academy = User.objects.create_user(
            username="academia",
            email="academia@example.com",
            password="TestPassword123!",
            is_academy=True,
        )
        self.student1 = User.objects.create_user(
            username="laura",
            email="laura@example.com",
            password="TestPassword123!",
            is_student=True,
        )
        self.student2 = User.objects.create_user(
            username="carlos",
            email="carlos@example.com",
            password="TestPassword123!",
```

```
            is_student=True,
        )

    def test_academy_can_create_group(self):
        conv = create_group_conversation(
            creator=self.academy,
            title="Alumnos",
            participant_ids=[self.student1.pk, self.student2.pk],
        )
        self.assertTrue(conv.is_group)
        self.assertEqual(conv.participants.count(), 3)

    def test_student_cannot_create_group(self):
        with self.assertRaises(PermissionDenied):
            create_group_conversation(
                creator=self.student1,
                title="Grupo no autorizado",
                participant_ids=[self.student2.pk],
            )


class SendMessageServiceTests(TestCase):

    def setUp(self):
        self.student = User.objects.create_user(
            username="laura",
            email="laura@example.com",
            password="TestPassword123!",
            is_student=True,
        )
        self.teacher = User.objects.create_user(
            username="profesor",
            email="profesor@example.com",
            password="TestPassword123!",
            is_academy=True,
        )
        self.conv = create_direct_conversation(self.student, self.teacher)

    def test_participant_can_send(self):
        msg = send_message(self.student, self.conv.pk, "Hola profe")
        self.assertEqual(msg.text, "Hola profe")
        self.assertEqual(msg.sender, self.student)

    def test_empty_message_raises(self):
        with self.assertRaises(ValidationError):
            send_message(self.student, self.conv.pk, "   ")

    def test_non_participant_cannot_send(self):
        intruder = User.objects.create_user(
            username="intruso",
            email="intruso@example.com",
```

```
            password="TestPassword123!",
        )
        with self.assertRaises(PermissionDenied):
            send_message(intruder, self.conv.pk, "Me cuelo")

    def test_mark_messages_as_read(self):
        msg = send_message(self.student, self.conv.pk, "Hola")
        mark_messages_as_read(self.conv.pk, self.teacher)
        msg.refresh_from_db()
        self.assertTrue(msg.is_read)
```

18.4. `chat/tests/test_views.py`

Archivo: `chat/tests/test_views.py` (crear).

Acción: crear archivo nuevo:

```
from django.contrib.auth import get_user_model
from django.test import TestCase
from django.urls import reverse

from chat.models import Conversation, Participant
from chat.services import create_direct_conversation

User = get_user_model()


class ChatWebViewTests(TestCase):

    def setUp(self):
        self.student = User.objects.create_user(
            username="laura",
            email="laura@example.com",
            password="TestPassword123!",
            is_student=True,
        )
        self.teacher = User.objects.create_user(
            username="profesor",
            email="profesor@example.com",
            password="TestPassword123!",
            is_academy=True,
        )
        self.conv = create_direct_conversation(self.student, self.teacher)

    def test_list_requires_login(self):
        response = self.client.get(reverse("chat:list"))
        self.assertEqual(response.status_code, 302)

    def test_list_shows_conversations(self):
```

```
        self.client.force_login(self.student)
        response = self.client.get(reverse("chat:list"))
        self.assertEqual(response.status_code, 200)

    def test_detail_requires_participant(self):
        intruder = User.objects.create_user(
            username="intruso",
            email="intruso@example.com",
            password="TestPassword123!",
        )
        self.client.force_login(intruder)
        response = self.client.get(reverse("chat:detail", kwargs={"pk": self.conv.pk}))
        self.assertEqual(response.status_code, 403)

    def test_detail_shows_messages(self):
        self.client.force_login(self.student)
        response = self.client.get(reverse("chat:detail", kwargs={"pk": self.conv.pk}))
        self.assertEqual(response.status_code, 200)

    def test_post_sends_message(self):
        self.client.force_login(self.student)
        response = self.client.post(
            reverse("chat:detail", kwargs={"pk": self.conv.pk}),
            {"text": "Hola"},
        )
        self.assertEqual(response.status_code, 302)
        self.assertEqual(self.conv.messages.count(), 1)
```

18.5. `chat/tests/test_api.py`

Archivo: `chat/tests/test_api.py` (crear).

Acción: crear archivo nuevo:

```
from django.contrib.auth import get_user_model
from rest_framework import status
from rest_framework.test import APITestCase

from chat.services import create_direct_conversation

User = get_user_model()


class ChatAPITests(APITestCase):

    def setUp(self):
        self.student = User.objects.create_user(
            username="laura",
            email="laura@example.com",
```

```
            password="TestPassword123!",
            is_student=True,
        )
        self.teacher = User.objects.create_user(
            username="profesor",
            email="profesor@example.com",
            password="TestPassword123!",
            is_academy=True,
        )
        self.conv = create_direct_conversation(self.student, self.teacher)

    def test_list_requires_auth(self):
        response = self.client.get("/api/v1/conversations/")
        self.assertIn(
            response.status_code,
            [status.HTTP_401_UNAUTHORIZED, status.HTTP_403_FORBIDDEN],
        )

    def test_list_shows_conversations(self):
        self.client.force_authenticate(user=self.student)
        response = self.client.get("/api/v1/conversations/")
        self.assertEqual(response.status_code, status.HTTP_200_OK)

    def test_create_direct(self):
        other = User.objects.create_user(
            username="carlos",
            email="carlos@example.com",
            password="TestPassword123!",
            is_student=True,
        )
        self.client.force_authenticate(user=self.student)
        response = self.client.post(
            "/api/v1/conversations/create_direct/",
            {"username": "carlos"},
            format="json",
        )
        self.assertEqual(response.status_code, status.HTTP_201_CREATED)

    def test_send_message(self):
        self.client.force_authenticate(user=self.student)
        response = self.client.post(
            f"/api/v1/conversations/{self.conv.pk}/send/",
            {"text": "Hola"},
            format="json",
        )
        self.assertEqual(response.status_code, status.HTTP_201_CREATED)

    def test_mark_read(self):
        self.client.force_authenticate(user=self.teacher)
        response = self.client.post(
            f"/api/v1/conversations/{self.conv.pk}/mark_read/",
```

```
        )
        self.assertEqual(response.status_code, status.HTTP_200_OK)
```

---

19. Tests — notifications

19.1. `notifications/tests/test_tasks.py`

Archivo: `notifications/tests/test_tasks.py` (crear).

Acción: crear archivo nuevo:

```
from django.contrib.auth import get_user_model
from django.test import TestCase

from notifications.models import Notification
from notifications.tasks import (
    create_notification_async,
    send_notification_email_async,
)

User = get_user_model()


class NotificationTaskTests(TestCase):

    def setUp(self):
        self.user = User.objects.create_user(
            username="laura",
            email="laura@example.com",
            password="TestPassword123!",
        )

    def test_create_notification_async(self):
        result = create_notification_async.delay(
            user_id=self.user.pk,
            notification_type=Notification.Type.INFO,
            title="Hola",
            message="Mensaje de prueba",
            url="/",
        )
        self.assertEqual(Notification.objects.filter(user=self.user).count(), 1)

    def test_send_email_async(self):
        result = send_notification_email_async.delay(
            user_id=self.user.pk,
            subject="Test",
            body="Cuerpo del email",
```

```
        )
        self.assertIn(str(self.user.pk), result.result)
```

19.2. `notifications/tests/test_signals.py`

Archivo: `notifications/tests/test_signals.py` (crear).

Acción: crear archivo nuevo:

```
from decimal import Decimal

from django.contrib.auth import get_user_model
from django.test import TestCase

from academies.models import Academy
from chat.services import create_direct_conversation, send_message
from courses.models import Category, Convocatoria, Course
from enrollments.models import Enrollment
from notifications.models import Notification

User = get_user_model()


class MessageSignalTests(TestCase):

    def setUp(self):
        self.student = User.objects.create_user(
            username="laura",
            email="laura@example.com",
            password="TestPassword123!",
            is_student=True,
        )
        self.teacher = User.objects.create_user(
            username="profesor",
            email="profesor@example.com",
            password="TestPassword123!",
            is_academy=True,
        )
        self.conv = create_direct_conversation(self.student, self.teacher)

    def test_new_message_creates_notification(self):
        send_message(self.student, self.conv.pk, "Hola profe")
        self.assertTrue(
            Notification.objects.filter(
                user=self.teacher,
                type=Notification.Type.NEW_MESSAGE,
            ).exists()
        )
```

```python
class EnrollmentSignalTests(TestCase):

    def setUp(self):
        self.owner = User.objects.create_user(
            username="academia",
            email="academia@example.com",
            password="TestPassword123!",
            is_academy=True,
        )
        self.student = User.objects.create_user(
            username="laura",
            email="laura@example.com",
            password="TestPassword123!",
            is_student=True,
        )
        self.academy = Academy.objects.create(
            owner=self.owner,
            name="English Now",
            slug="english-now",
        )
        self.category = Category.objects.create(name="Idiomas", slug="idiomas")
        self.course = Course.objects.create(
            academy=self.academy,
            category=self.category,
            title="Inglés B2",
            slug="ingles-b2",
            description="...",
            status="published",
        )
        self.convocatoria = Convocatoria.objects.create(
            course=self.course,
            start_date="2026-07-01",
            end_date="2026-09-30",
            price=Decimal("480.00"),
            total_places=15,
        )

    def test_new_enrollment_creates_notification_for_academy(self):
        Enrollment.objects.create(
            student=self.student,
            convocatoria=self.convocatoria,
        )
        self.assertTrue(
            Notification.objects.filter(
                user=self.owner,
                type=Notification.Type.ENROLLMENT_RECEIVED,
            ).exists()
        )
```

20. Verificación final

Acción: en terminal.

```
python manage.py check
python manage.py migrate
python manage.py test chat
python manage.py test notifications
python manage.py test
```

Todo debe pasar sin errores.

Arrancar el servidor con WebSockets:

```
python manage.py runserver
```

Daphne se encarga automáticamente.

Arrancar el worker de Celery (en otra terminal):

```
celery -A config worker --loglevel=info --pool=solo
```

En Windows, `--pool=solo` es necesario.

---

21. Checklist

Chat

- [x] App `chat` creada.
- [x] Modelos `Conversation`, `Participant`, `Message`.
- [x] Servicios `create_direct_conversation`, `create_group_conversation`, `send_message`, `mark_messages_as_read`.
- [x] Permiso `IsParticipant`.
- [x] Serializers (Conversation, Message, Participant, Inputs).
- [x] Vistas web (listado, detalle, crear grupo, chat directo).
- [x] Vistas REST (`ConversationViewSet` con actions).
- [x] WebSocket consumer (`ChatConsumer`).
- [x] Routing WS.
- [x] Templates (listado, detalle, crear grupo).
- [x] JS de tiempo real.
- [x] Admin.

- [x] Tests (modelos, servicios, vistas, API).

Notifications

- [x] Ampliación de `Notification.Type`.
- [x] Tareas Celery.
- [x] Señales `post_save` (Message, Enrollment, Scholarship, Course, Payment).
- [x] `apps.py` con `ready()`.
- [x] Tests de tareas y señales.

Configuración

- [x] `channels`, `channels-redis`, `daphne`, `celery` instalados.
- [x] `settings/base.py` con Channels, Celery, ASGI.
- [x] `settings/testing.py` con in-memory y eager.
- [x] `config/asgi.py` reescrito.
- [x] `config/celery.py` creado.
- [x] `config/__init__.py` modificado.
- [x] `config/urls.py` con `chat/`.
- [x] `api/urls.py` con `conversations`.

---

22. Registro de sesión

```
PROYECTO: Academy Hub
SESIÓN: 10B + 11B
FASE: Comunicación, notificaciones y tareas asíncronas
TEMA: Chat (REST + WebSockets), notificaciones por señales, Celery

OBJETIVO:
- Implementar chat entre alumno↔profesor, academia↔alumno, academia↔profesor,
  con soporte de grupos, en tiempo real (WebSockets) y por API REST.
- Añadir Celery con Redis como broker para tareas asíncronas.
- Desacoplar notificaciones de vistas mediante señales Django.

TEORÍA:
- Django Channels
- Channel Layer con Redis
- ASGI
- WebSockets
- Consumers
- Signals (pre_save, post_save)
```

```
- Celery (shared_task, broker, worker)
- Redis como broker
- Transacciones atómicas y select_for_update

APPS TOCADAS:
- chat (nueva)
- notifications (ampliada)
- config (settings, asgi, celery)
- api (router)
- templates y static

TESTS:
- chat: modelos, servicios, vistas web, API
- notifications: tareas, señales

COMMIT:
feat: add real-time chat, notifications via signals and Celery tasks

ESTADO:
[ ] Pendiente
[ ] En progreso
[ ] Completada

SIGUIENTE SESIÓN:
Sesión 12 — (según PDF)
```

Notas finales para no romper nada

1. `chat/views.py` (REST) y `chat/views_web.py` (web) están separados a propósito. No mezclar.
2. `base.html` necesita tener `{% block scripts %}{% endblock %}` antes de `</body>`. Si no lo tiene, añádelo.
3. `static/chat/js/` es una carpeta nueva. Créala.
4. Si `User` no tiene `is_teacher`, el código de `create_group_conversation` hace fallback a `is_academy` o `is_staff`. No rompe.
5. Si el proyecto no usa `runserver` con Daphne, hay que arrancar con `daphne config.asgi:application` o `python -m daphne config.asgi:application` en lugar de `runserver`. Con `channels` instalado, `runserver` usa Daphne automáticamente.
6. Los tests de señales usan `CELERY_TASK_ALWAYS_EAGER = True`, así que las tareas se ejecutan en línea y las notificaciones se crean dentro del mismo test.

7. El `Channel Layer` en testing es `InMemoryChannelLayer`, así que `send_message` puede llamar a `group_send` sin Redis levantado.